

lecture_10

August 24, 2026

1 Lecture 10: Transportation Engineering Problems - I

{note} Lectures 1-9 treated linear programs as abstract collections of decision variables and constraints. Many transportation problems, however, have an underlying network structure - nodes connected by arcs, with flow moving from origins to destinations. This special structure does not change the mathematics (network problems are still LPs, solvable by everything we have built so far), but it changes how we formulate and interpret them. This lecture covers two foundational network problems - the **Maximum Flow Problem** and the **Least-Cost Path Problem**. Lecture 11 continues with the **Transportation Problem** and the **Transshipment Problem**.

Learning Objectives

By the end of this notebook, you will be able to: 1. Represent a transportation network as a graph of nodes and arcs, and translate it into an LP formulation for the Maximum Flow Problem and the Least-Cost Path Problem. 2. Solve network LPs using Excel Solver and Python SciPy. 3. Interpret shadow prices, reduced costs, and duality results in a network context (capacity bottlenecks, cut sets, redundant arcs) to make informed routing and capacity-planning decisions.

Prerequisites: LP Formulation (Lecture 3), Excel Solver (Lecture 5), Python SciPy Method (Lecture 6), Sensitivity Analysis (Lecture 8), Duality (Lecture 9)

Estimated time: 90 minutes (including in-class exercises)

1.1 Why Transportation Engineering Problems?

Many transportation engineering questions share a common shape: a set of locations (depots, junctions, terminals) connected by routes (roads, rail links, shipping lanes), each with some capacity or cost, and the question is how to move people, vehicles, or goods through this network as efficiently as possible.

- MTC asks: *what is the maximum number of buses we can relocate per hour from our depots to the central hub, given road and yard capacity limits?* — a **Maximum Flow Problem**.
- NHAI asks: *which sequence of highway links gets a convoy from Chennai to Bengaluru at the lowest total travel cost?* — a **Least-Cost Path Problem**.

Both questions could be written as a generic LP and solved with the BFS table (Lecture 7) or `linprog` (Lecture 6) exactly as before — nothing new is required mathematically.

Note on scope: This course always formulates and solves transportation engineering problems through their LP formulation (Excel Solver, Python SciPy), not specialised network algorithms (e.g. Ford-Fulkerson, Dijkstra). The LP view keeps everything consistent with sensitivity analysis and duality from Lectures 8-9, and it generalises cleanly to constraints that specialised algorithms cannot easily handle.

1.2 Network Notation

We will use the following notation throughout Lectures 10-11:

Symbol	Meaning
$\mathcal{N}$	Set of nodes (junctions, depots, terminals)
$\mathcal{A}$	Set of directed arcs (roads, links, routes), $\mathcal{A} \subseteq \mathcal{N} \times \mathcal{N}$
$(i, j) \in \mathcal{A}$	A directed arc from node i to node j
u_{ij}	Capacity of arc (i, j) — maximum flow it can carry
c_{ij}	Cost (or length, or travel time) of arc (i, j)
x_{ij}	Decision variable — flow sent along arc (i, j)
o	Source / origin node
d	Sink / destination node

A network is drawn as a **directed graph**: circles (nodes) connected by arrows (arcs), each arc labelled with its capacity and/or cost.

1.3 The Maximum Flow Problem

1.3.1 Problem Statement

Given a network with a single **source** node o and a single **sink** node d , and each arc $(i, j) \in \mathcal{A}$ carrying a capacity u_{ij} , find the largest total flow that can be sent from o to d without violating any arc capacity or breaking flow conservation at intermediate nodes.

1.3.2 LP Formulation

Decision variables: x_{ij} = flow sent along arc (i, j) , for every $(i, j) \in \mathcal{A}$.

Objective: maximize total flow leaving the source (equivalently, total flow arriving at the sink),

$$\max \quad z = \sum_{j:(o,j) \in \mathcal{A}} x_{oj}$$

Subject to:

$$\begin{aligned}
\text{(Capacity)} \quad & 0 \leq x_{ij} \leq u_{ij} \quad \forall (i, j) \in \mathcal{A} \\
\text{(Flow conservation)} \quad & \sum_{i: (i, j) \in \mathcal{A}} x_{ij} - \sum_{k: (j, k) \in \mathcal{A}} x_{jk} = 0 \quad \forall j \in \mathcal{N} \setminus \{o, d\}
\end{aligned}$$

{note} Flow conservation is the defining constraint of every network-flow problem we will study in Lectures 10-11: at any node that is not a source or sink, flow in must equal flow out - nothing is created, stored, or destroyed at an intermediate node. Only the source and sink are exempt; the source is a net producer of flow and the sink a net consumer.

This is a linear program exactly as in Lecture 3 — decision variables, a linear objective, and linear constraints. The only thing “network” about it is the structure of the constraint matrix: each flow-conservation row has only +1s and −1s (one per arc touching that node), and each capacity constraint involves a single variable.

1.4 In-Class Exercises

1.4.1 Exercise 1

Problem Statement BMTC (Bengaluru Metropolitan Transport Corporation) needs to relocate idle buses from its Yelahanka depot (o) to the Majestic central hub (d) before the evening peak. Buses can only travel along approved routes between depots/junctions, and each route has a maximum number of buses it can carry per hour (limited by road width, signal timings, and traffic mixed with regular service).

The approved route network (junction $\rightarrow$ junction, capacity in buses/hour) is:

Arc	From	To	Capacity (buses/hr)
1	Yelahanka (o)	Hebbal (H)	8
2	Yelahanka (o)	Yeshwantpur (Y)	5
3	Hebbal (H)	Majestic (d)	6
4	Hebbal (H)	Yeshwantpur (Y)	3
5	Yeshwantpur (Y)	Majestic (d)	7

Maximize the number of buses relocated per hour from Yelahanka to Majestic.

Step 1 — Formulate the LP Let $x_{oH}, x_{oY}, x_{Hd}, x_{HY}, x_{Yd}$ denote hourly bus flow on each of the five arcs.

$$\max \quad z = x_{oH} + x_{oY}$$

$$\begin{aligned}
(\text{Capacity}) \quad & x_{oH} \leq 8, \quad x_{oY} \leq 5, \quad x_{Hd} \leq 6, \quad x_{HY} \leq 3, \quad x_{Yd} \leq 7 \\
(\text{Flow conservation at H}) \quad & x_{oH} = x_{Hd} + x_{HY} \\
(\text{Flow conservation at Y}) \quad & x_{oY} + x_{HY} = x_{Yd} \\
& x_{oH}, x_{oY}, x_{Hd}, x_{HY}, x_{Yd} \geq 0
\end{aligned}$$

Note: We maximize $z = x_{oH} + x_{oY}$ (flow leaving the source). By flow conservation, this automatically equals $x_{Hd} + x_{Yd}$ (flow arriving at the sink) — we do not need to write both as separate objectives.

Step 2 — Excel Spreadsheet Layout

	A	B	C	D	E	F	
1	BMTc BUS RELOCATION - MAXIMUM FLOW						
2							
3	Arc	o-H	o-Y	H-d	H-Y	Y-d	
4	Capacity (buses/hr)	8	5	6	3	7	
5							
6	Decision Variables	o-H	o-Y	H-d	H-Y	Y-d	
7	Flow (buses/hr)	0	0	0	0	0	← Solver changes these
8							
9	Objective						
10	Total Flow	=B7+C7					
11							
12	Constraints	LHS		RHS	Slack		
13	Cap o-H	=B7		8	=C13-B13		
14	Cap o-Y	=C7		5	=C14-B14		
15	Cap H-d	=D7		6	=C15-B15		
16	Cap H-Y	=E7		3	=C16-B16		
17	Cap Y-d	=F7		7	=C17-B17		
18	Conservation @ H	=B7-D7-E7		0	← equality		
19	Conservation @ Y	=C7+E7-F7		0	← equality		

Step 3 — Solver Configuration

- **Set Objective:** B10 → Max
- **Variable Cells:** B7:F7
- **Constraints:**
 - B13 C13
 - B14 C14
 - B15 C15
 - B16 C16
 - B17 C17
 - B18 = C18
 - B19 = C19

- B7:F7 0
- **Method:** Simplex LP → Solve

Step 4 — Expected Results & Interpretation

Arc	Optimal flow (buses/hr)
x_{oH}	8 (at capacity)
x_{oY}	5 (at capacity)
x_{Hd}	6 (at capacity)
x_{HY}	2
x_{Yd}	7 (at capacity)
Total Flow	13 buses/hr

Managerial insight: Three arcs are fully saturated — $o \rightarrow H$, $H \rightarrow d$, and $Y \rightarrow d$ — meaning every available slot on these routes is used. The arc $H \rightarrow Y$ carries only 2 of its available 3 buses/hr — it has spare capacity but is constrained by what flow conservation at H allows ($x_{oH} = 8$ splits into $x_{Hd} = 6$ at capacity and the remaining 2 routed via Y).

1.5 The Least-Cost Path Problem

1.5.1 Problem Statement

Given a network with arc costs c_{ij} (travel time, fuel + toll cost, distance — any additive impedance), find the path from source o to sink d with the smallest total cost.

1.5.2 LP Formulation

The least-cost path problem is modelled as sending **one unit of flow** from o to d at minimum cost — the LP “discovers” the cheapest path because flow conservation forces it to choose a connected sequence of arcs.

Decision variables: $x_{ij} \geq 0$ = flow (fraction of the unit shipment) on arc (i, j) , for every $(i, j) \in \mathcal{A}$.

Objective: minimize total cost,

$$\min \quad z = \sum_{(i,j) \in \mathcal{A}} c_{ij} x_{ij}$$

Subject to:

$$\text{(Supply at source)} \quad \sum_{j:(o,j) \in \mathcal{A}} x_{oj} - \sum_{j:(j,o) \in \mathcal{A}} x_{jo} = 1$$

$$\text{(Flow conservation)} \quad \sum_{i:(i,j) \in \mathcal{A}} x_{ij} - \sum_{k:(j,k) \in \mathcal{A}} x_{jk} = 0 \quad \forall j \in \mathcal{N} \setminus \{o, d\}$$

$$x_{ij} \in \{0, 1\} \quad \forall (i, j) \in \mathcal{A}$$

{tip} Unlike Maximum Flow, arcs in a least-cost path problem are typically left uncapacitated ($u_{ij} = \infty$) – the question is not "how much can flow", but "which route is cheapest". With $\mathbf{c} \geq 0$ and one unit of supply at o , the optimal LP solution is always integer (0 or 1 on each arc) and traces out a single path, even though we never explicitly told the LP to pick "a path" rather than splitting flow.

1.5.3 Exercise 2

Problem Statement NHAI is advising a freight convoy on the cheapest route from Chennai (o) to Bengaluru (d), via two possible intermediate junctions: Vellore (A) and Krishnagiri (B). The “cost” of each highway link captures fuel, toll, and time, combined into a single ‘00/trip figure.

Arc	From	To	Cost (‘00/trip)
1	Chennai (o)	Vellore (A)	3
2	Chennai (o)	Krishnagiri (B)	5
3	Vellore (A)	Krishnagiri (B)	1
4	Vellore (A)	Bengaluru (d)	6
5	Krishnagiri (B)	Bengaluru (d)	2

Minimize total trip cost from Chennai to Bengaluru.

Step 1 — Formulate the LP Let $x_{oA}, x_{oB}, x_{AB}, x_{Ad}, x_{Bd}$ denote the flow (fraction of one trip) carried on each arc.

$$\min \quad z = 3x_{oA} + 5x_{oB} + 1x_{AB} + 6x_{Ad} + 2x_{Bd}$$

$$\text{(Supply at } o) \quad x_{oA} + x_{oB} = 1$$

$$\text{(Conservation at A)} \quad x_{AB} + x_{Ad} - x_{oA} = 0$$

$$\text{(Conservation at B)} \quad x_{Bd} - x_{oB} - x_{AB} = 0$$

$$\text{(Demand at } d) \quad x_{Ad} + x_{Bd} = 1$$

$$x_{oA}, x_{oB}, x_{AB}, x_{Ad}, x_{Bd} \in \{0, 1\}$$

Step 2 — Solve with `scipy.optimize.linprog` Recall from Lecture 6: `linprog` minimizes by default, equality constraints go in `A_eq/b_eq`, and `res.eqlin.marginals` returns the shadow prices of equality constraints (Lecture 8).

```
[1]: # --- Exercise 2: solve the NHAI least-cost path LP ---

import numpy as np
from scipy.optimize import linprog

c = [3, 5, 1, 6, 2]                # objective coefficients: cost on each arc

# Equality constraints: A_eq @ x = b_eq
# Row 1 - supply at o:           x_oA + x_oB                = 1
# Row 2 - conservation at A:     -x_oA      + x_AB + x_Ad      = 0
# Row 3 - conservation at B:           -x_oB - x_AB      + x_Bd = 0
# Row 4 - demand at d:                               x_Ad + x_Bd = 1
A_eq = [
    [ 1,  1,  0,  0,  0],
    [-1,  0,  1,  1,  0],
    [ 0, -1, -1,  0,  1],
    [ 0,  0,  0,  1,  1],
]
b_eq = [1, 0, 0, 1]                # right-hand side: supply/demand at each node
bounds = [(0, None)] * 5          # all flows non-negative, uncapacitated

res = linprog(c, A_eq=A_eq, b_eq=b_eq, bounds=bounds, method='highs')

arc_names = ["x_oA", "x_oB", "x_AB", "x_Ad", "x_Bd"]
print("Optimal flows:")
for name, val in zip(arc_names, res.x):
    print(f"    {name} = {val:.2f}")
print(f"\nMinimum cost z* = {res.fun*100:,.0f}/trip")
```

Optimal flows:

```
x_oA = 1.00
x_oB = 0.00
x_AB = 1.00
x_Ad = 0.00
x_Bd = 1.00
```

Minimum cost $z^* = 600/\text{trip}$

Optimal solution: $x_{oA}^* = 1$, $x_{AB}^* = 1$, $x_{Bd}^* = 1$ (all other arcs zero), $z^* = 6$ (600/trip).

The LP has selected the path **Chennai** → **Vellore** → **Krishnagiri** → **Bengaluru**, at total cost 600/trip — cheaper than going directly via Krishnagiri (5+ 2 = 700) or directly via Vellore (3+ 6 = 900).

Step 3 — Sensitivity Analysis: Reduced Costs

```
[10]: # --- Exercise 2: reduced costs ---
print("\nReduced costs (non-basic arcs):")
for name, val in zip(arc_names, res.lower.marginals):
    print(f"    {name}: reduced cost = {val:.2f}")
```

Reduced costs (non-basic arcs):

```
x_oA: reduced cost = 0.00
x_oB: reduced cost = 1.00
x_AB: reduced cost = 0.00
x_Ad: reduced cost = 3.00
x_Bd: reduced cost = 0.00
```

Reading the output: {note} ****Reduced cost as a network quantity**:** for any arc (i,j) , the reduced cost equals $\bar{c}_{ij} = c_{ij} - (u_i - u_j)$. An arc that is part of the optimal path always has $\bar{c}_{ij}=0$ (verify: x_{oA} , x_{AB} , x_{Bd} all show reduced cost 0 above). An arc not used has $\bar{c}_{ij}>0$, and this value tells us exactly how much the arc's cost would need to fall before it becomes attractive.

Arc (unused)	Reduced cost	Interpretation
x_{oB} (Chennai $\rightarrow$ Krishnagiri direct)	1	This link would need to get 100/trip <i>cheaper</i> (cost 4) before NHAI would prefer it over routing via Vellore
x_{Ad} (Vellore $\rightarrow$ Bengaluru direct)	3	This link would need to get 300/trip cheaper (cost 3) before it joins the optimal route

Managerial insight: NHAI does not need to re-solve the whole LP every time a toll changes. If the Chennai–Krishnagiri toll is reduced by less than 100/trip, the optimal route through Vellore remains unchanged — the reduced cost tells the planner exactly how much “slack” exists before the recommended route would shift.

1.6 Take-Away Exercises

Work through each exercise following these steps: 1. Draw the network. 2. Formulate the LP (decision variables, objective, constraints). 3. Solve using the specified tool (Excel Solver or Python SciPy). 4. Where indicated, carry out the sensitivity/duality analysis requested. 5. Record the optimal solution and write a one-paragraph managerial interpretation.

1.6.1 Exercise 1

The Mundra Port (Gujarat) is evacuating containers from its Port Yard (o) to the Rail Yard (d) ahead of a vessel arrival, routed through two intermediate transfer terminals, T1 and T2. Each link has a maximum hourly container-handling capacity:

Arc	From	To	Capacity (containers/hr)
1	Port Yard (o)	T1	5
2	Port Yard (o)	T2	8
3	T1	T2	4
4	T1	Rail Yard (d)	7
5	T2	Rail Yard (d)	9

Maximize the number of containers evacuated per hour from the Port Yard to the Rail Yard.

Use Excel Solver.

1.6.2 Exercise 2

Pune's freight network draws inbound trucks from two source depots, Chakan (S1) and Hinjawadi (S2), into the central Pune Depot (d), partly via an intermediate Hub (H) and partly via a direct bypass link from Chakan.

Arc	From	To	Capacity (trucks/hr)
1	Chakan (S1)	Hub (H)	6
2	Hinjawadi (S2)	Hub (H)	9
3	Chakan (S1)	Pune Depot (d)	3
4	Hub (H)	Pune Depot (d)	8

Maximize total trucks/hr arriving at the Pune Depot from both sources combined.

Use Python SciPy.

1. Identify which arcs are at full capacity (binding) at the optimum.
 2. The set of binding arcs that, if removed, would completely disconnect the sources from the sink is called a **cut**. Verify that the total capacity of the binding arcs you identified equals the maximum flow — this is the **Max-Flow Min-Cut Theorem**, the network-flow special case of LP strong duality (Lecture 9): *the maximum flow from source to sink always equals the minimum capacity of any cut separating them.*
-

1.6.3 Exercise 3

A Chennai-based last-mile delivery operator must route a delivery van from its Hub (o) to a Customer Cluster (d), via two possible waypoints, A and B. Each road segment carries a cost (in 10s, combining fuel and time):

Arc	From	To	Cost (10/trip)
1	Hub (o)	A	4
2	Hub (o)	B	7
3	A	B	2
4	A	Customer Cluster (d)	5
5	B	Customer Cluster (d)	2

Minimize total delivery trip cost from the Hub to the Customer Cluster.

Use Excel Solver.

1.6.4 Exercise 4

A commuter on Hyderabad's Outer Ring Road (ORR) corridor wants the cheapest (in generalised travel cost) route from an Origin Junction (o) to a Destination Junction (d), via two ORR way-points, A and B:

Arc	From	To	Cost (/trip)
1	Origin (o)	A	5
2	Origin (o)	B	8
3	A	B	2
4	A	Destination (d)	7
5	B	Destination (d)	4

Minimize total generalised travel cost from Origin to Destination.

Use Python SciPy.

1. Report the reduced cost of every unused arc.
2. For each unused arc, state how much its cost would need to fall before it enters the optimal route (recall Lecture 8's objective-coefficient ranging, applied here to arc costs instead of generic c_j).

1.7 Circling Back

- **Lecture 3 (LP Formulation), Lecture 5 (Excel Solver), Lecture 6 (Python SciPy):** every network problem in this lecture is solved using exactly the formulation and solver skills already built — the only new ingredient is the graph representation.
- **Lecture 8 (Sensitivity Analysis):** shadow prices on capacity constraints in a Maximum Flow problem identify bottleneck arcs; reduced costs on flow variables in a Least-Cost Path problem become node potentials, with a clean physical meaning (shortest distance from the source).
- **Lecture 9 (Duality):** the dual of the Maximum Flow LP is the Minimum Cut problem — strong duality here is the classical Max-Flow Min-Cut Theorem, explored in Take-Away Exercise 2.

1.8 Moving Forward

- **Lecture 11 (Transportation Engineering Problems - II)**: extends this same network-LP toolkit to two more transportation-relevant structures — the **Transportation Problem** (multiple sources, multiple destinations, no intermediate nodes) and the **Transshipment Problem** (sources, destinations, *and* intermediate trans-shipment points). Flow conservation remains the unifying idea throughout.
-

1.9 Further Reading

- Ahuja, R.K., Magnanti, T.L., and Orlin, J.B. (1993). *Network Flows: Theory, Algorithms, and Applications* — Chapters 1–6 (max flow), Chapter 4 (shortest path).
- Winston, W.L. (2004). *Operations Research: Applications and Algorithms* — Chapter 8: Network Models.
- Vanderbei, R.J. (2014). *Linear Programming: Foundations and Extensions* — Chapter 14: Network Flow Problems.
- SciPy documentation: `scipy.optimize.linprog` — [docs.scipy.org](https://docs.scipy.org/doc/scipy/reference/optimize/linprog.html)
- networkx documentation: networkx.org